



Московский государственный университет имени М.В.Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра автоматизации систем вычислительных комплексов

Варгин Артём Анатольевич

Алгоритм управления перегрузкой на основе потерь с использованием методов прямого исправления ошибок

Курсовая работа

Научный руководитель:
Ассистент, к.ф.-м.н.
Степанов Евгений Павлович

Москва, 2025

Аннотация

Алгоритм управления перегрузкой на основе потерь с использованием методов прямого исправления ошибок

Варгин Артём Анатольевич

В работе рассматривается задача повышения скорости передачи транспортного потока при помощи использования методов прямого исправления ошибок (FEC) в алгоритмах управления перегрузкой на основе потерь (TCP-CUBIC). Проведен обзор методов прямого исправления ошибок, применяемых для исправления заданного количества ошибок стирания. Был предложен алгоритм управления перегрузкой с применением следующих алгоритмов прямого исправления ошибок: 1-PR, 2-PR, R-код и код Рида-Соломона с быстрым преобразованием Фурье. Предложенный алгоритм реализован в среде имитационного моделирования ns-3. Проведено экспериментальное исследование полученного алгоритма с целью сравнения скорости транспортного потока под управлением предложенного алгоритма управления перегрузки и под управлением TCP-CUBIC.

Содержание

1	Введение	5
1.1	Изменение требований к сетевым сервисам	5
1.2	Проблемы TCP	6
1.3	Мотивация использования FEC	7
1.4	Цель и задачи работы	8
1.5	Структура работы	9
2	Постановка задачи	10
3	Обзор существующих методов FEC	11
3.1	1-PR	12
3.2	2-PR	12
3.3	R-код	13
3.4	PC-код	14
3.5	Вывод	15
4	Описание решения	17
4.1	Общий подход к применению FEC в TCP	17
4.2	Проблема повторной передачи	18
4.2.1	Модификация задержки подтверждений	20
4.3	Особые случаи	20
4.4	Выбор размера блока N	20
5	Программная реализация	22
6	Экспериментальное исследование эффективности полученной реализации относительно стандартного TCP CUBIC	24
6.1	Цель исследования	24
6.2	Методика проведения экспериментального исследования	24

6.3	Анализ результатов	28
7	Заключение	32
	Список литературы	33

1 Введение

Рост Интернета, обусловленный развитием инфраструктуры и увеличением доступности широкополосных соединений [14], стал катализатором для появления широкого спектра сетевых сервисов, включая мультимедийные платформы (например, YouTube, Netflix) [15], онлайн-банкинг и трейдинг (PayPal, Robinhood) [16], социальные сети (Facebook, Twitter) [17], а также массовые многопользовательские онлайн-игры (World of Warcraft, Fortnite) [18]. Изначально Интернет разрабатывался для передачи символьных данных [13], с помощью сервисов, таких как электронная почта.

С развитием сетевых технологий появление высокоскоростных широкополосных сетей (например, сотовых сетей 5G, беспроводных сетей IEEE 802.11ac) и терабитных оптоволоконных линий передачи данных не заставило себя долго ждать. Стало возможным предоставление мультимедийных сервисов, чувствительных к задержкам (например, потоковое видео). Развитие технологий 5G/6G и edge computing [19] создает основу для массовой поддержки в ближайшем десятилетии сервисов, критичных к задержкам, включая автономные автомобили [20] и системы смешанной реальности (Microsoft Mesh, Meta Quest) [21].

1.1 Изменение требований к сетевым сервисам

С развитием Интернета изменились требования к сетевым сервисам. Особенно возрос спрос на сервисы реального времени, которые мгновенно обрабатывают поступившие события или запросы. Качество таких сервисов зависит не только от скорости передачи данных, но и, в значительной степени, от задержек. Скорость передачи была улучшена благодаря развитию широкополосных сетей, снижению задержек обработки на устройствах, снижению задержек пакетизации и т.д., однако дальнейший прогресс в снижении задержек ограничен расстоянием между узлами связи. Поэтому требуется повышение способности протокола максимально использовать доступную пропускную способность сети, минимизируя при этом задержки, потери пакетов и избыточный трафик. Несмотря на изменения требований к сетевым сервисам, протокол TCP [14]

по-прежнему остаётся основным протоколом для надёжной передачи данных. Несмотря на адаптацию транспортных протоколов к современным требованиям — внедрение алгоритмов управления перегрузкой (BBR, CUBIC) — сохраняются фундаментальные ограничения:

- Физические задержки, обусловленные расстоянием (например, RTT между континентами ≥ 80 мс [22]);
- Конфликт между гарантией доставки (TCP) и низкой задержкой (актуально для онлайн-игр и AR/VR [23]);
- Накладные расходы протоколов (TLS-рукопожатие, служебные заголовки), снижающие полезную пропускную способность [24].

1.2 Проблемы TCP

Протокол TCP управляет перегрузкой, динамически регулируя скорость передачи данных. Когда сеть перегружена (например, из-за переполнения буферов маршрутизаторов), пакеты начинают теряться. TCP интерпретирует эти потери как сигнал перегрузки и снижает скорость передачи, чтобы уменьшить нагрузку. Потерянные пакеты переотправляются, а скорость остаётся ограниченной до тех пор, пока сеть не стабилизируется. Для восстановления потерянных пакетов требуется минимум один RTT. В средах с высокой задержкой время повторной передачи увеличивается, что снижает эффективность TCP. Пример изменения скорости передачи (размера окна перегрузки, `cwnd`) показан на Рис. 1. Из-за реактивного управления перегрузкой (снижения скорости передачи после обнаружения потерь пакетов) TCP плохо подходит для задач, требующих минимальных задержек. Чтобы решить проблему, необходимо минимизировать количество повторных передач.

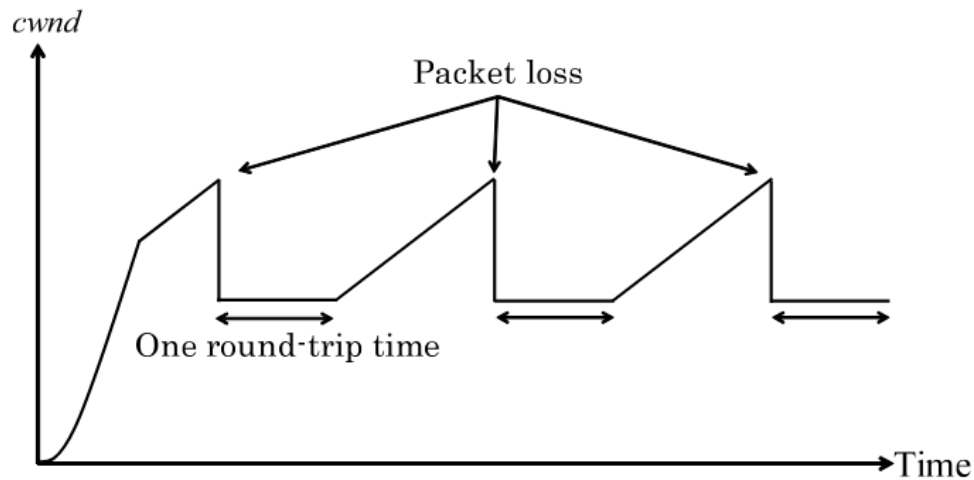


Рис. 1: Изменение размера окна перегрузки [3]

1.3 Мотивация использования FEC

Один из способов предотвращения потерь — использование методов прямого исправления ошибок (FEC)[7]. FEC позволяет отправителю передавать пакеты с избыточной информацией, чтобы получатель мог восстановить потерянные данные. Успешность восстановления зависит от объёма избыточности, однако это создаёт дополнительную нагрузку на сеть. Технология чаще используется в UDP (с постоянной скоростью передачи), но плохо совместима с TCP, где скорость передачи изменчива [3]. Пример работы FEC с TCP показан на Рис. 2:

1. Отправитель (S) отправляет 3 пакета: 2 информационных (P1 и P2) и 1 избыточный (F1);
2. В процессе передачи теряется информационный пакет P2;
3. Получатель (Re) восстанавливает потерянный пакет P2, используя избыточный пакет F1.

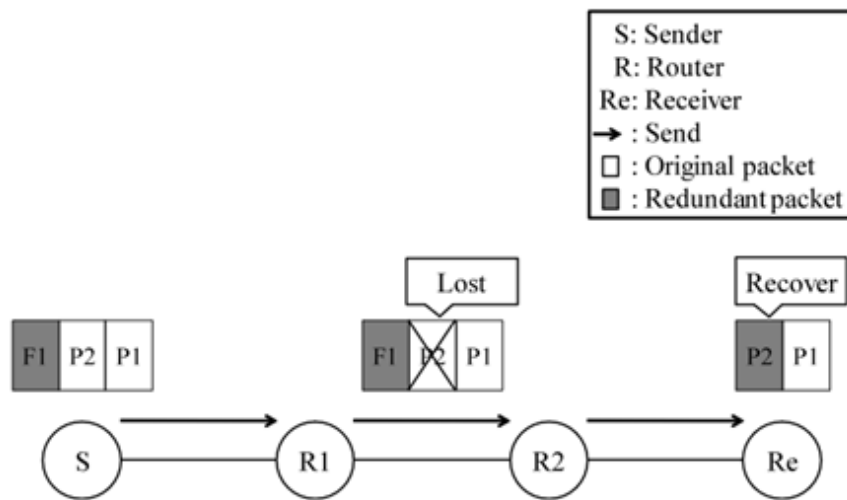


Рис. 2: Пример использования FEC [3]

1.4 Цель и задачи работы

Цель: Разработка алгоритма управления перегрузкой на основе TCP CUBIC с использованием механизма Forward Error Correction (FEC), направленного на повышение скорости передачи данных в наложенном канале.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести обзор методов FEC;
2. Разработка алгоритма управления перегрузкой на основе потерь с использованием FEC;
3. Проведение экспериментального исследования эффективности и справедливости разработанного алгоритма.

1.5 Структура работы

В главе 2 описывается постановка задачи. В главе 3 проводится обзор существующих методов FES. В главе 4 приводится метод добавления избыточности в TCP и рассматривается ряд проблем, возникающих при его интегрировании. В главе 5 приводится краткая сводка о программной реализации алгоритма в ns3. В главе 6 представлены результаты экспериментального исследования полученного алгоритма с целью сравнения скорости транспортного потока под управлением предложенного алгоритма управления перегрузкой и под управлением TCP-CUBIC. Глава 7 подводит заключение настоящей работы.

2 Постановка задачи

Дано:

1. Алгоритм управления перегрузкой TCP CUBIC
2. Наложенный канал, описываемый:
 - Уровнем потерь $error_p$. Считаем, что случайный процесс потери пакета описывается законом Бернулли (т.е. количество потерь произошедших в блоке размерности N: $X \sim Bi(N, error_p)$);
 - Задержкой $delay$.
3. Конечное упорядоченное множество пакетов $P = \{p_1, p_2, \dots, p_n\}$.

Необходимо:

1. Разбить поток на блоки FEC длины N. N выбирается такое, что максимизирует скорость транспортного потока.
2. Предложить операцию отправки следующего пакета на отправителе и условие отправки подтверждения на получателе, взяв за основу алгоритм управления перегрузкой TCP CUBIC.

3 Обзор существующих методов FEC

Forward Error Correction – это метод помехоустойчивого кодирования, основная идея которого состоит во введении избыточности в передаваемые данные с помощью кода, исправляющего ошибки стирания.

Модуль FEC состоит из 2 элементов:

- Кодировщик - модуль, отвечающий за формирование блока FEC на отправителе, т.е. создание избыточных пакетов на основе входящих в данный блок информационных;
- Декодировщик - модуль, отвечающий за восстановление потерянных пакетов на стороне получателя.

Методы будем сравнивать по следующим критериям:

1. Количество исправляемых ошибок - определяет устойчивость метода к потерям пакетов в условиях перегрузки сети. Чем больше ошибок может исправить алгоритм, тем выше надежность передачи данных, но тем больше избыточности требуется;
2. Сложность кодирования - влияет на задержку при подготовке данных к отправке. Высокая сложность увеличивает нагрузку на отправителя, что критично для систем реального времени (например, видеотрансляций);
3. Сложность декодирования - определяет требования к вычислительным ресурсам получателя;
4. Количество избыточных пакетов - накладные расходы сети. Избыточность улучшает устойчивость к потерям, но сокращает полезную пропускную способность, что может усугубить перегрузку.

Далее, при рассмотрении методов FEC будем опираться на материалы обзора [7] в предположении, что размеры пакетов одинаковые.

3.1 1-PR

Метод 1-PR добавляет к блоку информационных пакетов один пакет избыточности (проверочный пакет). На Рис. 3 проиллюстрирована схема работы метода 1-PR: биты проверочного пакета получают операцией XOR над битами одноимённых разрядов всех информационных пакетов. Если нет потерь пакетов, или потерян проверочный пакет, выход декодера — информационные пакеты блока. Если потерян ровно один пакет из блока, он восстанавливается поразрядным применением операции XOR к оставшимся пакетам. Иначе — отказ от декодирования, что приводит к запросу на повторную пересылку. Метод можно применять при произвольном размере блока.

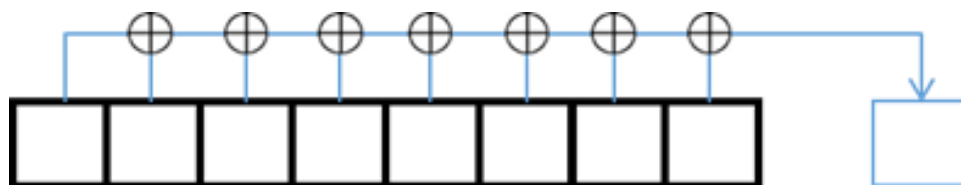


Рис. 3: 1-PR

3.2 2-PR

Метод 2-PR добавляет к последовательности информационных пакетов два проверочных пакета, биты которых получены применением операции XOR к содержимому одноимённых разрядов информационных пакетов, находящихся соответственно на чётных и нечётных местах в последовательности, составляющей блок (Рис. 4). Если нет потерь пакетов, или потеряны оба проверочных пакета, выход декодера — информационная часть блока. Если потеряны один пакет или два пакета последовательности в позициях разной чётности, они восстанавливаются, как в методе 1-PR. Иначе - отказ от декодирования, что приводит к запросу на повторную пересылку. Метод можно применять при произвольном размере блока.

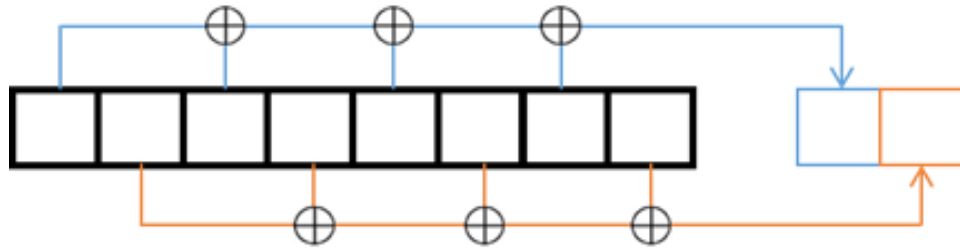


Рис. 4: 2-PR

3.3 R-код

Позволяет восстановить два потерянных информационных пакета, добавляя к последовательности из K информационных пакетов (И-часть) $M = \lfloor \log_2 K \rfloor + 2$ проверочных пакета (П-часть).

А. Кодирование. Пусть И-часть есть конечная последовательность $u = [u_1, \dots, u_K]$ пакетов $u_i, i = 1, \dots, K$. Тогда П-часть $s = [s_1, \dots, s_M]$ пачки формируется по правилу $s^T = Gu^T$.

У R-кода сложность операций кодирования составляет $O(K \log_2 K)$.

Б. Декодирование.

1. Если потеряно более двух пакетов, то происходит отказ от декодирования, что приводит к запросам на повторную пересылку.
2. Если потерь пакетов не произошло, или потерянными оказались только пакеты из П-части, то результат декодирования есть И-часть.
3. Если потерян один пакет u_j в И-части, а в П-части потерян не более, чем один пакет, то u_j восстанавливается по какому-то из сохранившихся пакетов чётности s_0 или s_{t+1} , как в методе 1-PR.
4. Пусть в И-части потеряны пакеты u_{j1} и u_{j2} , $1 \leq j1 < j2 \leq K$ (далее обозначение: $[k] = 1, 2, \dots, K$). Тогда

- (a) в подматрице G из двух столбцов j_1 и j_2 находим строку i вида $[01]$ (такая строка, возможно, не единственная, всегда найдётся).
- (b) восстанавливаем пакет u_{j_2} :
$$u_{j_2} = s_i + \sum_{j \in [k] \setminus s_{02}} g_{i,j} \cdot u_j$$
- (c) по пакету чётности s_1 восстанавливаем пакет u_{j_1} как в методе 1-PR.

Сложность операции декодирования у R-кода — линейная. Это следует из описания выше.

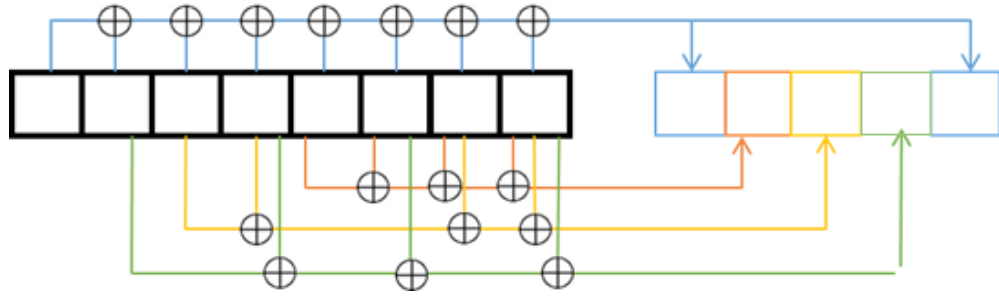


Рис. 5: R-код

3.4 РС-код

Пакет рассматриваем как многосимвольную конечную последовательность, каждый символ которой содержит m^* бит. Поэтому, оперируя символами, а не пакетами, при рассмотрении RS -кодов будем пользоваться обозначениями: код $RS[n, k]_q$ в поле Галуа $GF(q)$ из q элементов с количеством информационных символов k , проверочных избыточных — m и длиной $n = m + k$ (индекс q будем опускать).

Строим поле Галуа $GF(2^{m^*})$ как факторкольцо $F = F_2[x]/(a(x))$ кольца двоичных многочленов от формальной переменной x по идеалу неприводимого многочлена $a(x) \in F_2[x]$ степени $m^* = 12, 16$. В качестве $a(x)$ выбираем один из примитивных многочленов:

- при $m^* = 12 - a(x) = x^{12} + x^6 + x^4 + x + 1$,
- при $m^* = 16 - a(x) = x^{16} + x^{12} + x^3 + x + 1$.

Порождающую матрицу РС-кода в поле F строим, формируя порождающий полином $g(x)$.

А. Кодирование состоит в получении кодового слова v , которое получают умножением вектора u на порождающую матрицу G :

$$v = uG.$$

Б. Быстрое преобразование Фурье. Стоит отметить, что кодирование и декодирование кода Рида-Соломона при помощи решения СЛАУ имеет сложность $O(sm^3)$, т.е. порядка 1012 операций умножений и сложений символов в поле Галуа $GF(2^{16})$. Для работы на транспортном уровне такая сложность может быть неприемлема из-за создаваемой дополнительной задержки на вычисления.

Альтернативным подходом может стать использование быстрого преобразования Фурье. При таком подходе сложность кодирования и декодирования не превосходят $O(n \log n)$.

3.5 Вывод

Обзор содержит краткое представление о работе методов введения избыточности. Сравнение по перечисленным выше критериям представлено в таблице 1.

Считаем, что нам дан (n, k) код, где $m = n - k$ – число избыточных символов.

По результатам проведенного обзора были выбраны следующие методы введения избыточности: 1-PR, 2-PR, R-код и РС-код с бПФ.

Таблица 1: Сравнение характеристик кодов

Параметр	1-PR	2-PR	R-код	PC-код	PC-код с бПФ
Количество исправляемых ошибок	1	2 (при потерях на позициях разной чётности)	2	m	m
Сложность кодирования	$O(k)$	$O(k)$	$O(k \log k)$	$O(sm^3)$	$O(n \log n)$
Сложность декодирования	$O(k)$	$O(k)$	$O(k \log k)$	$O(sm^3)$	$O(n \log n)$
Количество избыточных пакетов	1	2	$2 + \lceil \log_2 k \rceil$	m	m

4 Описание решения

Прежде чем описать структуру алгоритма управления перегрузкой, необходимо проанализировать ключевые проблемы, выявленные в ходе разработки, и предложенные методы их устранения. Эти вопросы систематизированы в следующих разделах:

- Проблема эффективного использования метода FEC - минимизация количества повторных передач;
- Особые случаи, возникающие в процессе работы разработанного алгоритма;
- Проблема выбора размера блока FEC для различных методов прямого исправления ошибок.

4.1 Общий подход к применению FEC в TCP

Основная проблема добавления механизма прямого исправления ошибок в транспортный протокол заключается в выборе места, куда этот модуль должен встраиваться: например, кодировщик должен работать с *массивом байтов*, приходящими с уровня приложений или, наоборот, работать с уже сформированными пакетами, готовыми к передаче на уровень IP.

В работе [2] предлагается идея встраивания модуля FEC в часть транспортного уровня. Под этим подразумевается:

1. Кодировщик модуля прямого исправления ошибок (FEC) на стороне отправителя работает с пакетами TCP перед передачей их на уровень IP, (использует содержимое пакетов для формирования избыточного пакета) в блок FEC;
2. При заполнении блока (*количество пакетов в блоке* == N) сформированный избыточный пакет отправляется в сеть;
3. Декодировщик модуля FEC на стороне получателя формирует из последовательных пакетов отправленный блок FEC и при потерях запускается операция восстановления:

- Если количество потерянных пакетов меньше, чем уровень избыточности, то восстановление возможно, и на потерянный пакет отправляется подтверждение;
- Иначе - отказ от восстановления и запрос на переотправку.

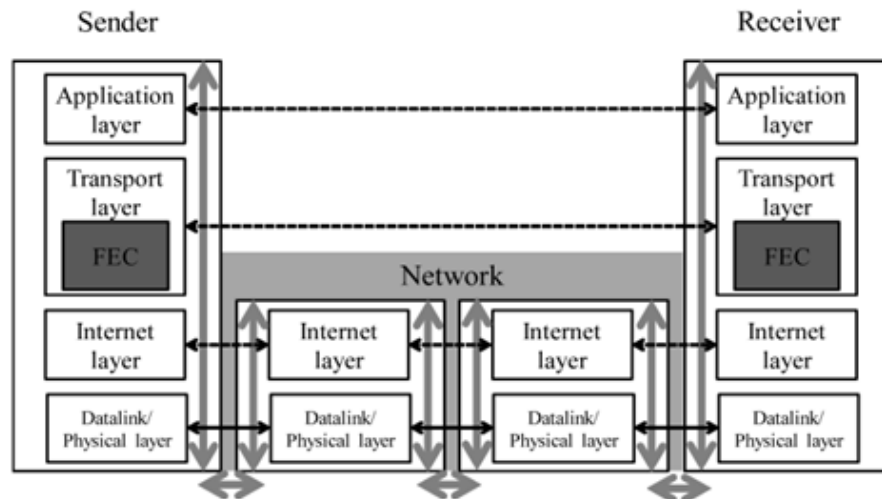


Рис. 6: Использование FEC в TCP [3]

4.2 Проблема повторной передачи

Использование механизма избыточности предполагает, что сначала будет произведена попытка восстановления потерянного пакета и, в случае её невозможности, отправка запроса на переотправку. Но данное требование не может быть выполнено без дополнительной модификации операции получения пакета на стороне получателя. На Рис.7 видно, что при потере первого пакета из блока FEC отправитель пошлёт запрос на переотправку даже в случае восстановления потерянного пакета.

Идея решения данной проблемы представлена в работе [2]. Оно заключается в задержке отправки подтверждений на полученные пакеты до момента получения послед-

него пакета из блока FEC. На стороне получателя:

- Декодировщик модуля FEC задерживает отправку подтверждений на полученные пакеты, до момента получения последнего пакета отправленного блока FEC;
- Декодировщик сравнивает количество потерянных пакетов в блоке с уровнем избыточности:
 - Если потерь пакетов нет - отправляются подтверждения на все пакеты блока;
 - Если потери были и восстановление возможно, то потерянный пакет восстанавливается и отправляются подтверждения на все пакеты блока;
 - Иначе отправляется запрос на повторную передачу потерянного пакета.

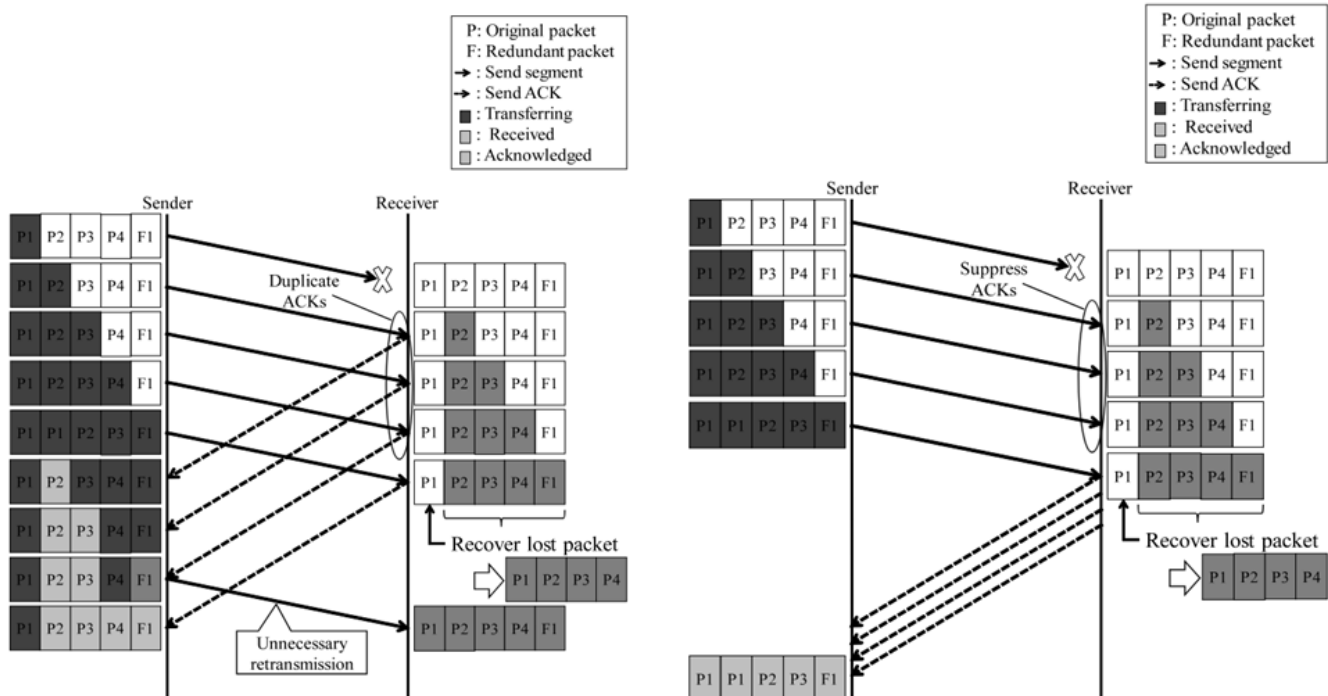


Рис. 7: Проблема повторной передачи [3]

4.2.1 Модификация задержки подтверждений

Для ускорения работы разработанного алгоритма было предложено модифицировать представленный в работе [2] алгоритм задержки отправки подтверждений: подтверждения начинают задерживаться только после первой потери пакета в отправленном блоке FEC, иначе работа алгоритма остается аналогичной стандартному механизму TCP.

4.3 Особые случаи

Под *особыми случаями* понимаются состояния, при которых формирование блока FEC может существенно уменьшить скорость транспортного потока:

1. Размер блока FEC N меньше чем $cwnd$.
 - Использование $N = cwnd$.
2. Невозможность формирования блока FEC из-за недостатка информационных пакетов:
 - Блок FEC является последним - содержит в себе последний пакет трафика:
 - Использование N равное количеству оставшихся пакетов.

4.4 Выбор размера блока N

В работе [11] была получена оценка средней скорости транспортного потока TCP CUBIC в зависимости от уровня потерь на канале и задержки:

$$R_{avg} = \frac{h}{error_p^d * RTT^e},$$
 где RTT - задержка в оба конца, $error_p$ - уровень потерь на канале, $e = 0.25, d = 0.75, h = 0.25$

В работе [7] были представлены таблицы оценки количества информационных пакетов K в блоке для обеспечения требуемого уровня потерь l .

Таким образом, итоговая формула средней полезной скорости транспортного потока TCP CUBIC с использованием FEC будет иметь вид:

$R_{res} = \frac{k}{n} \frac{h}{l^d * RTT^e}$, где член $\frac{k}{n}$ отвечает за подсчет только информационных пакетов.

5 Программная реализация

Работа над реализацией алгоритма управления перегрузкой проводилась в системе имитационного моделирования ns3-44.

В сборку были добавлены 2 класса *ForwardErrorCorrectionEncoder* и *ForwardErrorCorrectionDecoder* реализующие соответственно функции модуля FEC на стороне отправителя и получателя.

ForwardErrorCorrectionEncoder отвечает за:

- Добавление очередного пакета для создания избыточных пакетов (блок FEC состоит только из последовательных пакетов);
- Генерацию избыточных пакетов;

ForwardErrorCorrectionDecoder отвечает за:

- Добавление очередного пакета для проверки на возможность восстановления;
- Проверку заполнения блока;
- Восстановление потерянных пакетов;
- Задержку отправки подтверждений.

Также для того, чтобы отличать пакеты FEC, избыточные, от информационных, была добавлена опция FEC. Эта опция присваивалась информационному пакету в кодировщике на стороне отправителя.

Изменениям подверглись также модули изначальной сборки:

- *tcp – option* - для регистрации новой опции FEC;
- класс *TcpSocketBase* для модификации функций отправки пакетов, получения пакетов, отправки подтверждений в соответствии с идеями предложенными в пункте 4;

- Для корректной доставки пакетов с новой опцией FEC была изменена функция *ForwardUp* отвечающая за сбрасывание невалидных пакетов, которым относятся все пакеты с уникальными опциями.

6 Экспериментальное исследование эффективности полученной реализации относительно стандартного TCP CUBIC

6.1 Цель исследования

Необходимо сравнить скорости транспортного потока под управлением предложенного алгоритма управления перегрузкой для различных методов введения избыточности и под управлением TCP CUBIC.

6.2 Методика проведения экспериментального исследования

Тестирование будет проводиться с помощью средств системы имитационного моделирования ns3-44. Был взят за основу и доработан тест из [12]:

- Добавлены параметры командной строки, задающие уровень избыточности и размер блока;
- Тест был адаптирован под ns3-44.

Тестовый стенд представляет собой систему "точка-точка-точка-точка с трафиком, генерируемым отправителем с помощью *ApplicationContainer*.

Результаты экспериментов

Во-первых, требуется вычислить значение оптимального размера блока FEC для каждого метода кодировки и затем проверить это экспериментально. Для метода 1-PR, основываясь на работе [7], был подсчитан оптимальный размер блока FEC:

- Для вероятности потерь 0.01 оптимальное значение размера блока - 7.
- Для вероятности потерь 0.02 оптимальное значение размера блока - 3.
- Для вероятности потерь 0.03 - 0.04 оптимальное значение размера блока - 2.

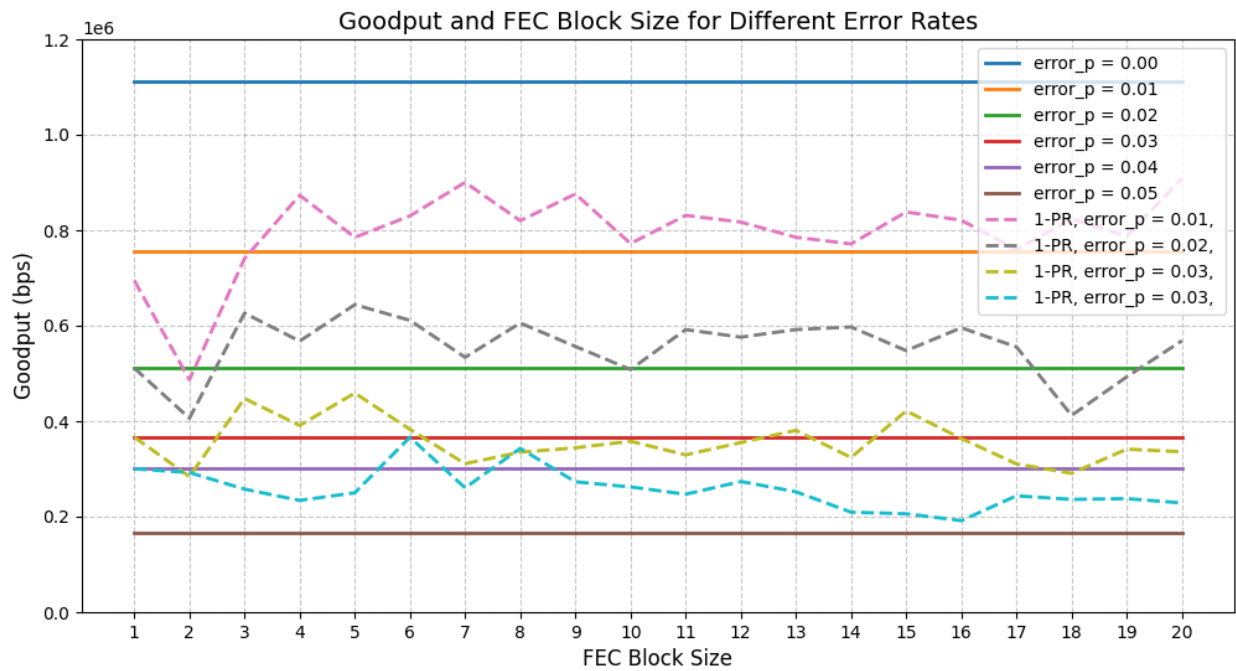


Рис. 8: Скорость передачи данных при различных значениях уровня потерь и размера блока для метода 1-PR

Для метода 2-PR, основываясь на работе [7], был подсчитан оптимальный размер блока FEC:

- Для вероятности потерь 0.01 оптимальное значение размера блока - 14.
- Для вероятности потерь 0.02 оптимальное значение размера блока - 4.
- Для вероятности потерь 0.02 оптимальное значение размера блока - 3.
- Для вероятности потерь 0.04 оптимальное значение размера блока - 2.

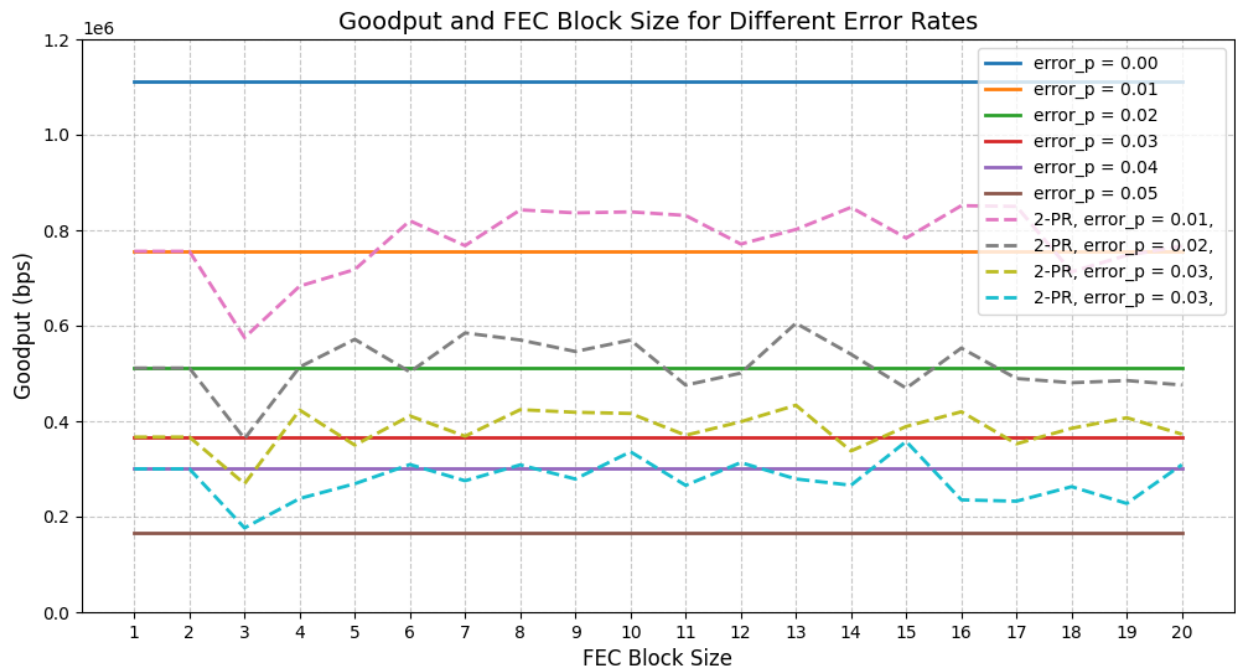


Рис. 9: Скорость передачи данных при различных значениях уровня потерь и размера блока для метода 2-PR

Для метода R-кода, основываясь на работе [7], было подсчитано оптимальное количество информационных пакетов в блоке FEC:

- Для вероятности потерь 0.01 оптимальное количество информационных пакетов в блоке - 16.
- Для вероятности потерь 0.02 оптимальное количество информационных пакетов в блоке - 6.
- Для вероятности потерь 0.03 оптимальное количество информационных пакетов в блоке - 4.

- Для вероятности потерь 0.04 оптимальное количество информационных пакетов в блоке - 2.

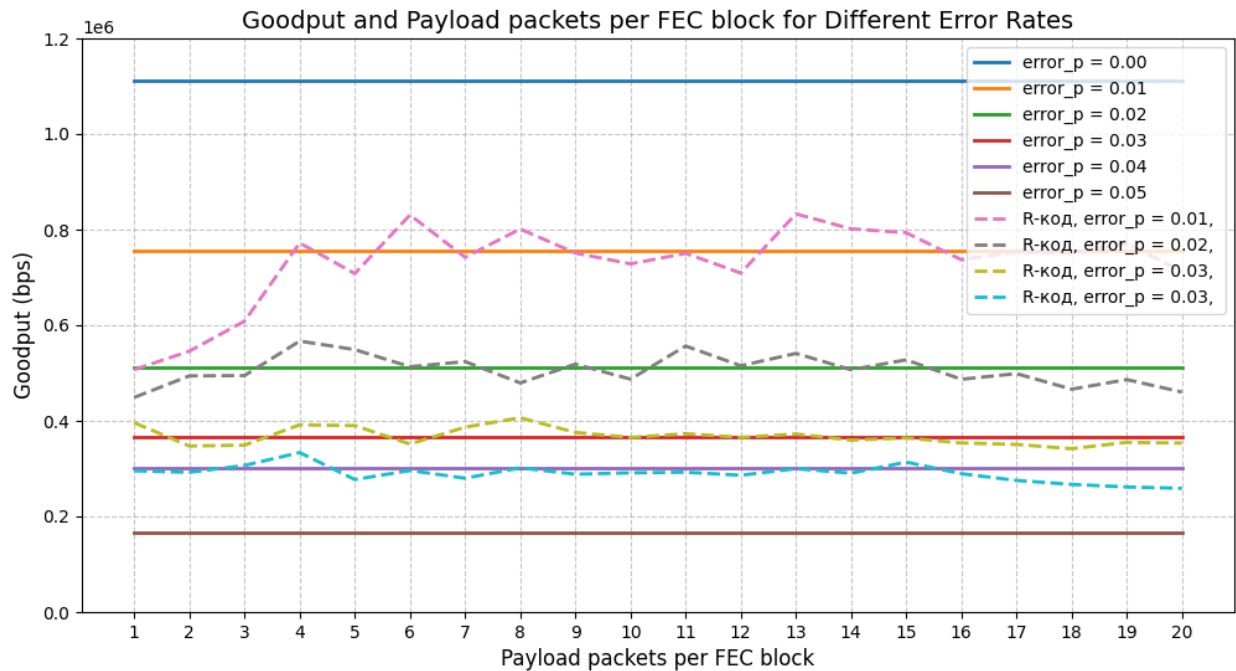


Рис. 10: Скорость передачи данных при различных значениях уровня потерь и количества информационных пакетов в блоке для R-кода

Для метода РС-кода, основываясь на работе [7], было подсчитано оптимальное количество информационных пакетов в блоке FEC:

- Для вероятности потерь 0.01 оптимальное количество информационных пакетов в блоке - 18.
- Для вероятности потерь 0.02 оптимальное количество информационных пакетов в блоке - 8.

- Для вероятности потерь 0.02 оптимальное количество информационных пакетов в блоке - 5.
- Для вероятности потерь 0.04 оптимальное количество информационных пакетов в блоке - 2.

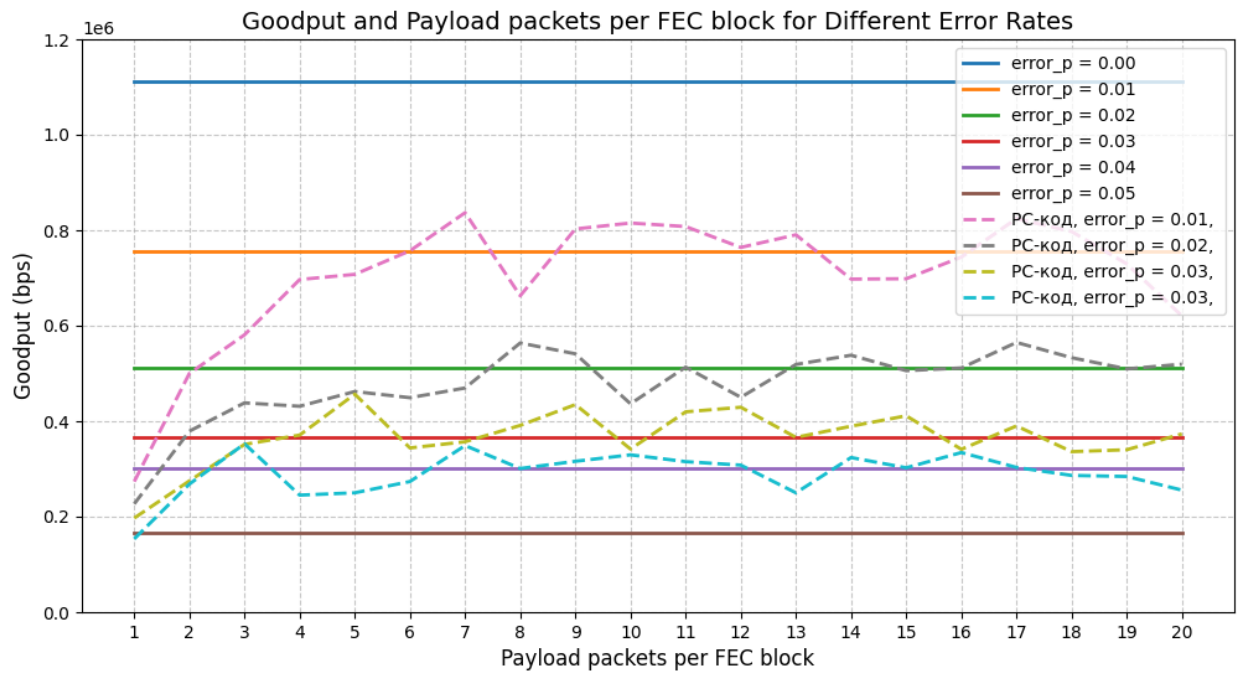


Рис. 11: Скорость передачи данных при различных значениях уровня потерь и количества информационных пакетов в блоке для РС-кода

6.3 Анализ результатов

На графиках Рис. 8 - 11 изображены результаты скорости транспортного потока под управлением разработанного алгоритма. Пунктирным линиям соответствуют скорости транспортного потока под управлением разработанного алгоритма при различных уровнях потерь, сплошные линии отвечают за скорости транспортного потока под

управлением TCP CUBIC при различном уровне потерь.

Рассмотрим каждый метод прямого исправления ошибок отдельно:

- **1-PR:** Для данного метода полученные выше значения размера блока оказались близкими к оптимальным.
 - Уровень потерь 0.01: скорость транспортного потока в среднем увеличилась не более чем на 10%;
 - Уровень потерь 0.02: скорость транспортного потока в среднем увеличилась не более чем на 10%;
 - Уровень потерь 0.03: увеличение скорости транспортного потока было только на близких к оптимальному значению размерах блока. В остальных случаях скорость либо не увеличивалась, либо становилась меньше;
 - Уровень потерь 0.04: увеличение скорости транспортного потока было только на близких к оптимальному значению размерах блока. В остальных случаях скорость либо не увеличивалась, либо становилась меньше;
- **2-PR:** Для данного метода полученные выше значения размера блока оказались близкими к оптимальным.
 - Уровень потерь 0.01: скорость транспортного потока в среднем увеличилась не более чем на 10%;
 - Уровень потерь 0.02: скорость транспортного потока в среднем увеличилась не более чем на 10%, но на размерах блока больших 16 скорость транспортного потока упала;
 - Уровень потерь 0.03: увеличение скорости транспортного потока было в пределах 10%;
 - Уровень потерь 0.04: скорости транспортного потока почти не отличается от стандартной реализации - увеличение точечное (потери пакетов происходили только на позициях разной четности);

- **R-код:** Для данного метода полученные выше значения размера блока оказались близкими к оптимальным.
 - Уровень потерь 0.01: скорость транспортного потока увеличилась больше всего при количестве информационных пакетов близком к оптимальным 16;
 - Уровень потерь 0.02: скорость транспортного потока увеличилась больше всего при количестве информационных пакетов близком к оптимальным 6. В среднем значение скорости транспортного потока было близко к скорости транспортного потока при стандартной реализации;
 - Уровень потерь 0.03: скорость транспортного потока увеличилась больше всего при количестве информационных пакетов близком к оптимальным 4. В среднем значение скорости транспортного потока было близко к скорости транспортного потока при стандартной реализации;
 - Уровень потерь 0.04: скорость транспортного потока увеличилась больше всего при количестве информационных пакетов близком к оптимальным 2. В среднем значение скорости транспортного потока было близко к скорости транспортного потока при стандартной реализации, но при количестве информационных пакетов ≥ 16 стала падать;
- **РС-код(4 избыточных пакета в блоке):** Для данного метода полученные выше значения размера блока оказались близкими к оптимальным. При малом (≤ 3) количестве пакетов во всех случаях скорость транспортного потока существенно уменьшалась из-за излишней избыточности.
 - Уровень потерь 0.01: скорость транспортного потока увеличилась больше всего при количестве информационных пакетов близком к оптимальным 18;
 - Уровень потерь 0.02: скорость транспортного потока увеличилась больше всего при количестве информационных пакетов близком к оптимальным 8. В среднем значение скорости транспортного потока было близко или чуть меньше скорости транспортного потока при стандартной реализации;

- Уровень потерь 0.03: скорость транспортного потока увеличилась больше всего при количестве информационных пакетов близком к оптимальным 5. В среднем значение скорости транспортного потока было близко к скорости транспортного потока при стандартной реализации;
- Уровень потерь 0.04: скорость транспортного потока увеличилась больше всего при количестве информационных пакетов близком к оптимальным 3. В среднем значение скорости транспортного потока было близко к скорости транспортного потока при стандартной реализации, но при количестве информационных пакетов ≥ 18 стала падать;

7 Заключение

В ходе выполнения настоящей работы были получены следующие результаты:

- Сделана постановка задачи разработки алгоритма управления перегрузок на основе потерь с использованием FEC.
- По результатам проведенного обзора методов введения избыточности было выбрано 4 метода: 1-PR, 2-PR, R-код и код РС с быстрым преобразованием Фурье,
- Разработан и реализован алгоритм управления перегрузок на основе потерь с использованием выбранных методов.
- Проведено сравнение скорости транспортного потока под управлением разработанного алгоритма управления перегрузкой и стандартным TCP CUBIC. По результатам исследования разработанный алгоритм показал увеличение данного показателя.

Дальнейший интерес для исследования представляют:

- Масштабирование экспериментов - усложнение схемы и проверка дружелюбности разработанного алгоритма.
- Модификация алгоритма за счет адаптивного выбора уровня избыточности.
- Модификация алгоритма для работы в условиях потерь, происходящих вспышками.

Список литературы

- [1] Complementing TCP Congestion Control with Forward Error Correction / Vicky Sharma, K. Ramakrishnan, Koushik Kar, Shivkumar Kalyanaraman. — Vol. 5550. — 2009. — 05. — Pp. 378–391.
- [2] SATO, Yurino. TCP Using Adaptive FEC to Improve Throughput Performance in High-Latency Environments / Yurino SATO, Hiroyuki KOGA, Takeshi Ikenaga // *IEICE Transactions on Communications*. — 2018. — 09. — Vol. E102.B.
- [3] Sato, Yurino. Improving TCP throughput using forward error correction / Yurino Sato, Hiroyuki Koga, Takeshi Ikenaga // *IEICE Communications Express*. — 2016. — 10. — Vol. 6.
- [4] Ковалёв, А. И. Анализ эффективности применения адаптивных механизмов управления перегрузкой для транспортных потоков / А. И. Ковалёв. — 2019.
- [5] Jacobson, V. Congestion avoidance and control / V. Jacobson // *ACM SIGCOMM Computer Communication Review*. — 2004. — 04. — Vol. 25. — Pp. 157–187.
- [6] Ha, S. CUBIC: A New TCP-Friendly High-Speed TCP Variant / S. Ha, I. Rhee, L. Xu // *Department of Computer Science, North Carolina State University*.
- [7] К вопросу восстановления пакетов на транспортном уровне / А. В. Борисов, С. И. Гуров, К. В. Семенихин et al. // *Вестник Московского университета. Серия 15. Вычислительная математика и кибернетика*. — 2024. — по. 7.
- [8] A Survey on Recent Advances in Transport Layer Protocols / M. Polese, F. Chiariotti, E. Bonetto et al. // *IEEE*.
- [9] Varma, Subir. Internet Congestion Control / Subir Varma. — 2015.
- [10] BBRv2 Evaluation in ns-3. — <https://github.com/lanigan23/BBRv2-Eval-ns-3.git>. — [Электронный ресурс].

- [11] *Hafner, K.* Where Wizards Stay Up Late: The Origins of the Internet / K. Hafner, M. Lyon. — New York: Simon & Schuster, 1996.
- [12] *Postel, J.* Transmission Control Protocol: RFC 793 / J. Postel: IETF, 1981.
- [13] *International Telecommunication Union (ITU).* Measuring Digital Development: Facts and Figures. — <https://www.itu.int/itu-d/reports>. — 2021. — [Электронный ресурс].
- [14] *Cisco Systems.* Visual Networking Index: Forecast and Trends, 2017–2022. — <https://www.cisco.com/vni>. — 2020.
- [15] *Bank for International Settlements (BIS).* Digital Banking and Fintech: Market Developments. — https://www.bis.org/publ/qtrpdf/r_qt1912e.htm. — 2019. — [Электронный ресурс].
- [16] *Pew Research Center.* Social Media Use in 2022. — <https://www.pewresearch.org/internet>. — 2022. — [Электронный ресурс].
- [17] *Taylor, T. L.* Watch Me Play: Twitch and the Rise of Game Live Streaming / T. L. Taylor. — Princeton University Press, 2018.
- [18] *IEEE.* Tech. Rep.: / IEEE: Institute of Electrical and Electronics Engineers, 2023.
- [19] *McKinsey & Company.* Autonomous Vehicles: Navigating the Next Decade. — <https://www.mckinsey.com/av2030>. — 2022. — [Электронный ресурс].
- [20] *Meta Platforms Inc.* The Future of Mixed Reality: Roadmap 2025–2030. — <https://about.meta.com/mr-roadmap>. — 2023. — [Электронный ресурс].
- [21] *International Telecommunication Union (ITU).* ITU Report on Global Latency. — <https://www.itu.int/reports>.
- [22] *Meta Platforms Inc.* Meta Quest Technical Requirements. — <https://www.meta.com/quest/specs>.

- [23] *Cloudflare*. Overhead of Modern Protocols. — <https://www.cloudflare.com/overhead>. — [Электронный ресурс].